



Basi di Dati
Progetto A.A. 2024/2025

PERSONAL TRAINER DIGITALE

0310751

Matteo Pinfildi

Indice

1. Descrizione del Minimondo.....	2
2. Analisi dei Requisiti	3
3. Progettazione concettuale.....	7
4. Progettazione logica	16
5. Progettazione fisica	24

1. Descrizione del Minimondo

1 Si vuole realizzare un'applicazione per consentire, ai personal trainer di una palestra, di
2 gestire le schede degli esercizi dei propri clienti.
3 Ciascun utente della palestra è associato ad un personal trainer di riferimento. Questi potrà
4 redigere una scheda di allenamento personalizzata per ciascun utente, scegliendo gli esercizi e
5 i macchinari da utilizzare in un insieme definito dal proprietario della palestra. Per ciascun
6 esercizio, il personal trainer indicherà anche il numero di serie e ripetizioni dell'esercizio.
7 Periodicamente, quando viene redatta una nuova scheda di esercizi, la scheda precedente
8 viene archiviata. Questa sarà ancora consultabile dagli utenti, ma essi potranno "interagire"
9 solo con quella corrente.
10 Gli utenti della palestra possono infatti accedere all'applicazione e mostrare quali sono gli
11 esercizi che devono svolgere in una sessione di allenamento. Il sistema mostrerà l'esercizio e
12 la serie corrente, permettendo all'utente di contrassegnare un dato esercizio come completato.
13 L'atleta ha la possibilità di saltare un esercizio, anche se cominciato, e passare al successivo.
14 I personal trainer possono generare un report che mostra, per tutti i clienti a loro assegnati,
15 quanti sono gli allenamenti sostenuti in un intervallo temporale richiesto, qual è la percentuale
16 di completamento delle schede di allenamento in ogni sessione di allenamento e quanto tempo
17 è durato ciascun allenamento.

2. Analisi dei Requisiti

Identificazione dei termini ambigui e correzioni possibili

Linea	Termine	Nuovo termine	Motivo correzione
2, 3, 4, 8, 10, 12, 14	Utente, Cliente	Atleta	Il nuovo termine si adatta meglio al contesto dell'allenamento, per cui è più specifico.
2, 7	Scheda degli esercizi, scheda	Scheda di allenamento	Specifica che stiamo parlando di un allenamento e non di esercizi in modo generico.
5	Proprietario della palestra	Gestore della palestra	Il nuovo termine indica meglio il ruolo organizzativo e decisionale relativo al proprietario della palestra.
9	Scheda corrente	Scheda di allenamento attiva	Specifica che stiamo parlando della scheda di allenamento utilizzata al momento.
7	Scheda precedente	Scheda di allenamento archiviata	Specifica che stiamo parlando di una scheda di allenamento già utilizzata e che ora non è più usata.
8	Interagire	Aggiornare	Specifica in modo più preciso l'azione che deve effettuare l'utente.
13	Saltare un esercizio	Esercizio successivo	L'atleta ha la possibilità di passare manualmente all'esercizio successivo anche se quello corrente non è stato completato.
15, 16	Percentuale di completamento	Progresso(%)	Termine più preciso e più orientato al mondo della palestra.
15, 17	Allenamento	Sessione di allenamento	E' troppo generico.

Specifica disambiguata

Si vuole realizzare un'applicazione per consentire, ai personal trainer di una palestra, di gestire le schede di allenamento dei propri atleti. Ciascun atleta della palestra è associato ad un personal trainer di riferimento. Questi potrà redigere una scheda di allenamento personalizzata per ciascun atleta, scegliendo gli esercizi e i macchinari da utilizzare in un insieme definito dal gestore della palestra. Per ciascun esercizio, il personal trainer indicherà anche il numero di serie e ripetizioni dell'esercizio. Periodicamente, quando viene redatta una nuova scheda di allenamento, la scheda di allenamento precedente viene archiviata. Questa sarà ancora consultabile dagli atleti, ma essi potranno aggiornare solo la scheda di allenamento attiva. Gli atleti della palestra possono infatti accedere all'applicazione e mostrare quali sono gli esercizi che devono svolgere in una sessione di allenamento. Il sistema mostrerà l'esercizio e la serie

corrente, permettendo al cliente di contrassegnare un dato esercizio come completato. L'atleta ha la possibilità di passare all'esercizio successivo, anche se ha già iniziato un esercizio. I personal trainer possono generare un report che mostra, per tutti gli atleti a loro assegnati, quanti sono le sessioni di allenamento sostenute in un intervallo temporale richiesto, qual è il progresso(%) delle schede di allenamento in ogni sessione di allenamento e quanto tempo è durato ciascuna sessione di allenamento.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
Atleta	Utente della palestra che accede all'app per visualizzare e seguire le schede di allenamento.	Utente, Cliente	Personal trainer, Sessione di allenamento, Scheda di allenamento
Personal trainer	Professionista che crea e gestisce le schede di allenamento per i clienti.		Atleta, Scheda di allenamento
Scheda di allenamento	Piano personalizzato contenente esercizi assegnati ad un cliente.	Scheda	Atleta, Esercizi, Personal trainer
Esercizi	Attività fisica specifica che si trova all'interno di una scheda di allenamento; ha serie e ripetizioni definite.		Scheda di allenamento, Sessione di allenamento, Macchinario
Macchinario	Elemento situato all'interno di una palestra per poter eseguire determinati esercizi.		Esercizi
Sessione di allenamento	Momento in cui un cliente della palestra usa la scheda corrente e svolge gli esercizi.	Allenamento	Esercizi, Atleta

Raggruppamento dei requisiti in insiemi omogenei

Frasi relative a Atleta

Ciascun atleta della palestra è associato ad un personal trainer di riferimento.

[...]

Periodicamente, quando viene redatta una nuova scheda di allenamento, la scheda di allenamento precedente viene archiviata. Questa sarà ancora consultabile dagli atleti, ma essi potranno aggiornare solo la scheda di allenamento attiva. Gli atleti della palestra possono infatti accedere all'applicazione e mostrare quali sono gli esercizi che devono svolgere in una sessione di allenamento.

[...]

L'atleta ha la possibilità di passare all'esercizio successivo, anche se ha già iniziato un esercizio.

Frasi relative a Personal trainer

Si vuole realizzare un'applicazione per consentire, ai personal trainer di una palestra, di gestire le schede di allenamento dei propri clienti.

Ciascun atleta della palestra è associato ad un personal trainer di riferimento. Questi potrà redigere una scheda di allenamento personalizzata per ciascun atleta, scegliendo gli esercizi e i macchinari da utilizzare in un insieme definito dal gestore della palestra. Per ciascun esercizio, il personal trainer indicherà anche il numero di serie e ripetizioni dell'esercizio.

[...]

I personal trainer possono generare un report che mostra, per tutti i clienti a loro assegnati, quanti sono le sessioni di allenamento sostenute in un intervallo temporale richiesto, qual è il progresso(%) delle schede di allenamento in ogni sessione di allenamento e quanto tempo è durato ciascuna sessione di allenamento.

Frasi relative a Scheda di allenamento

Periodicamente, quando viene redatta una nuova scheda di allenamento, la scheda di allenamento precedente viene archiviata. Questa sarà ancora consultabile dagli atleti, ma essi potranno aggiornare solo la scheda di allenamento attiva.

[...]

I personal trainer possono generare un report che mostra, per tutti gli atleti a loro assegnati, quanti sono le sessioni di allenamento sostenute in un intervallo temporale richiesto, qual è il progresso(%) delle schede di allenamento in ogni sessione di allenamento[...].

Frasi relative a Esercizi

Questi potrà redigere una scheda di allenamento personalizzata per ciascun atleta, scegliendo gli esercizi e i macchinari da utilizzare in un insieme definito dal gestore della palestra. Per ciascun esercizio, il personal trainer indicherà anche il numero di serie e ripetizioni dell'esercizio.

[...]

Gli atleti della palestra possono infatti accedere all'applicazione e mostrare quali sono gli esercizi che devono svolgere in una sessione di allenamento. Il sistema mostrerà l'esercizio e la serie corrente, permettendo al cliente di contrassegnare un dato esercizio come completato. L'atleta ha la possibilità di passare all'esercizio successivo, anche se ha già iniziato un esercizio.

Frasi relative a Macchinario

Questi potrà redigere una scheda di allenamento personalizzata per ciascun atleta, scegliendo gli esercizi e i macchinari da utilizzare in un insieme definito dal gestore della palestra.

Frasi relative a Sessione di allenamento

Gli atleti della palestra possono infatti accedere all'applicazione e mostrare quali sono gli esercizi che devono svolgere in una sessione di allenamento.

[...]

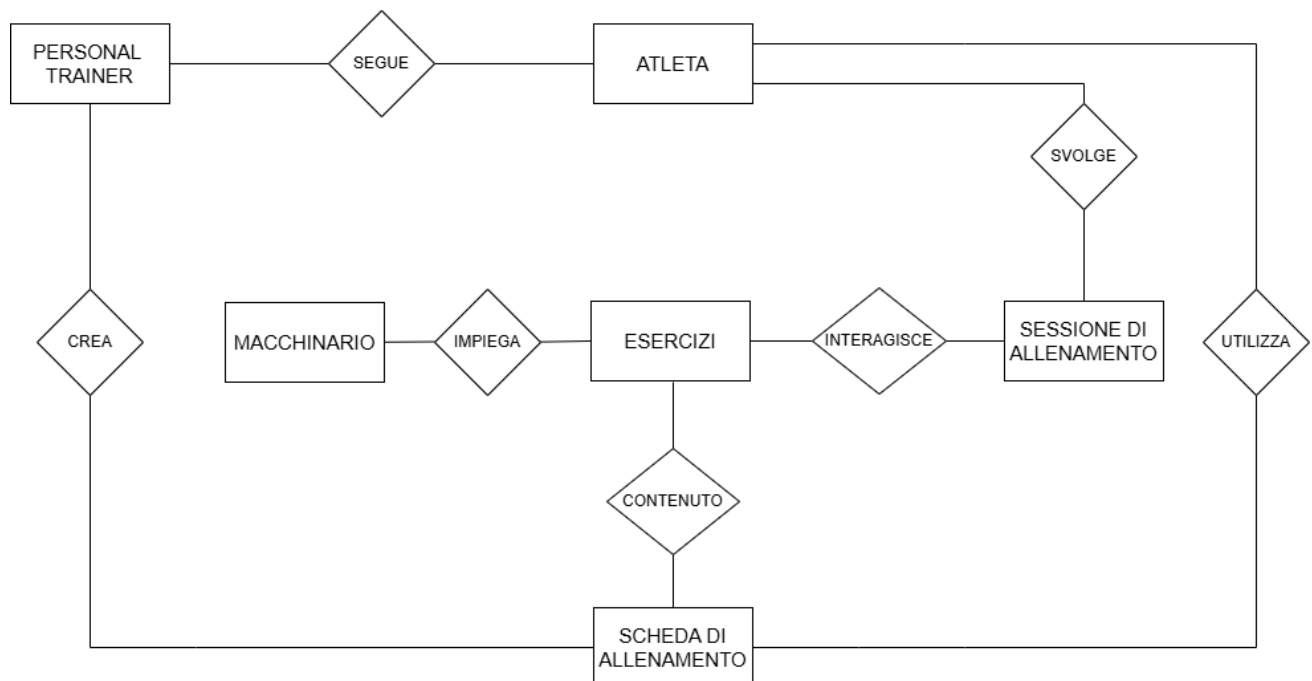
I personal trainer possono generare un report che mostra, per tutti gli atleti a loro assegnati, quanti sono le sessioni di allenamento sostenute in un intervallo temporale richiesto, qual è il progresso(%) delle schede di allenamento in ogni sessione di allenamento e quanto tempo è durato ciascuna sessione di allenamento.

3. Progettazione concettuale

Costruzione dello schema E-R

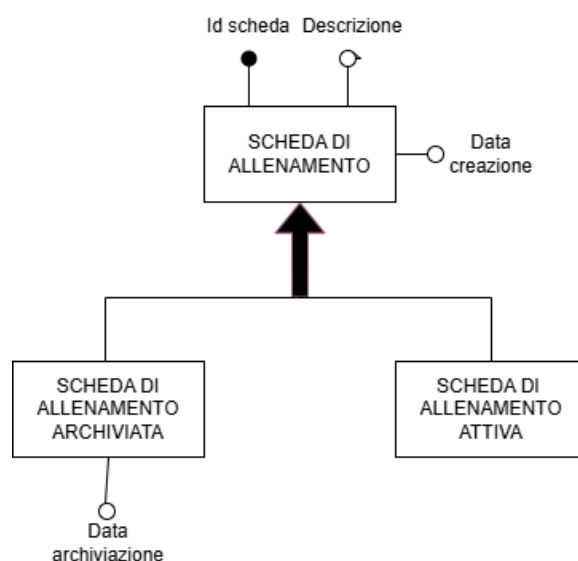
Passo 1: Definizione di uno schema scheletro.

Leggendo le specifiche è possibile delineare le entità e le relazioni principali che fanno parte dello schema concettuale. Si individuano le entità “Atleta”, “Personal trainer”, “Sessione di allenamento”, “Scheda di allenamento”, “Esercizi”, “Macchinario”, collegate tra di loro dalle relazioni mostrate in figura:

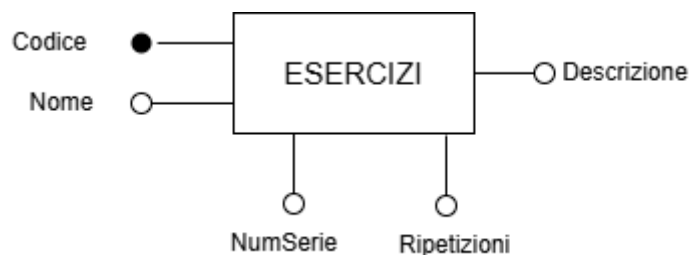


Passo 2: Raffinamento dell'entità "Scheda di allenamento".

Dalle specifiche raccolte possiamo notare che ci sono due tipi di schede di allenamento. Le "Schede di allenamento archiviate" e le "Schede di allenamento attive", possiamo rappresentare tale legame attraverso una generalizzazione totale ed esclusiva. Inoltre per ogni scheda di allenamento dobbiamo avere un identificativo, una descrizione della scheda e un attributo che indichi la data di creazione della scheda, mentre per le "Schede di allenamento archiviate", ci interessa la data di archiviazione. Possiamo inoltre notare, che dobbiamo aggiungere la relazione "Visualizza" tra l'entità "Atleta" e l'entità "Scheda di allenamento archiviata"; specificheremo meglio questo passaggio nei passi successivi. Lo schema intermedio dunque è:

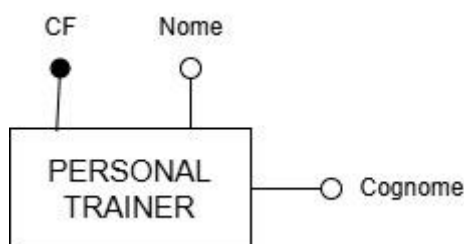
**Passo 3:** Raffinamento dell'entità "Esercizi".

All'entità "Esercizi", dobbiamo associargli i seguenti attributi: Codice (identificatore interno), Nome, Descrizione, NumSerie, Ripetizioni. Quindi abbiamo:

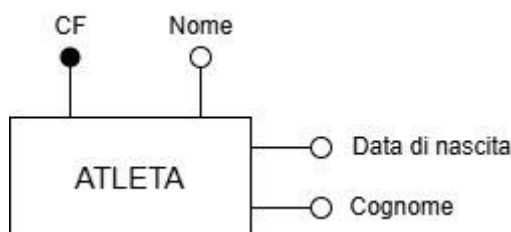


Passo 4: Raffinamento dell'entità "Personal trainer".

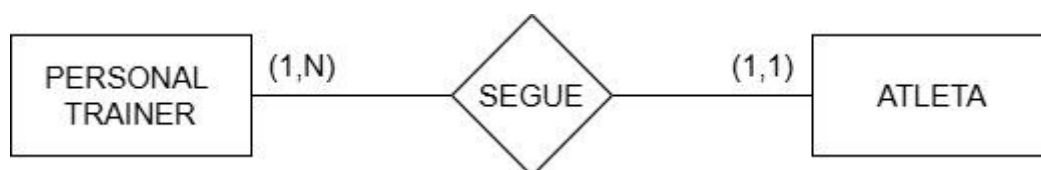
All'entità "Personal trainer" dobbiamo associargli i seguenti attributi: codice fiscale (che indicheremo con "CF", ed è identificatore interno), Nome e Cognome. Dunque:

**Passo 5:** Raffinamento dell'entità "Atleta".

All'entità "Atleta" dobbiamo associargli i seguenti attributi: il codice fiscale (che indicheremo con "CF") che costituisce l'identificatore dell'entità. Inoltre dobbiamo aggiungere gli attributi nome, cognome e data di nascita. Quindi avremo:

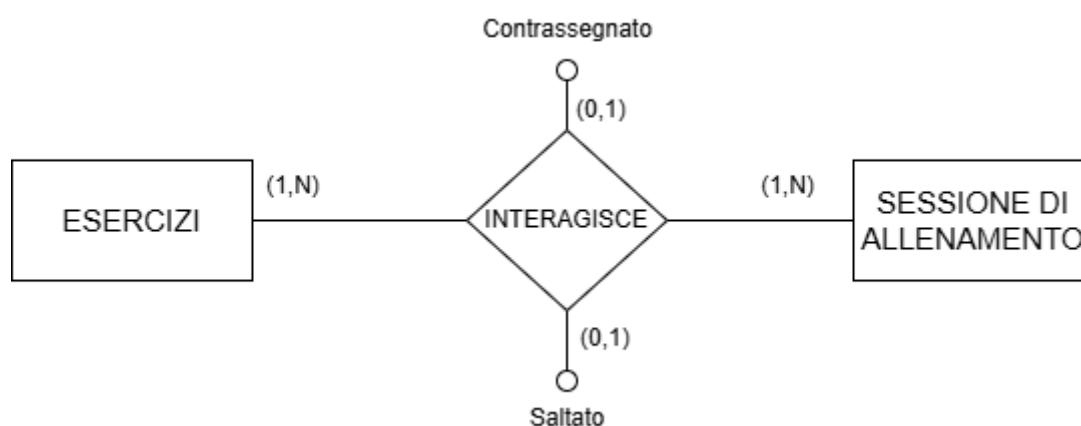
**Passo 6:** Cardinalità relazione "Segue".

L'entità "Personal trainer" partecipa alla relazione "Segue" con cardinalità (1,N), poiché un personal trainer può seguire più atleti, mentre l'entità "Atleta" partecipa alla relazione con cardinalità (1,1), dato che ogni atleta è associato ad un solo personal trainer. Per cui:



Passo 7: Cardinalità relazione “Interagisce”.

L’entità “Sessione di allenamento” partecipa alla relazione “Interagisce” con cardinalità (1,N), poiché in una sessione di allenamento si possono svolgere più esercizi, mentre l’entità “Esercizi” partecipa alla relazione con cardinalità (1,N), dato che un esercizio può essere svolto in più sessioni di allenamento. Inoltre la relazione “Interagisce” ha come attributo binario “Contrassegnato” per indicare se l’atleta ha svolto l’esercizio durante una specifica sessione di allenamento e l’attributo binario “Saltato” per indicare che l’atleta vuole passare all’esercizio successivo durante una specifica sessione di allenamento. Per cui avremo:

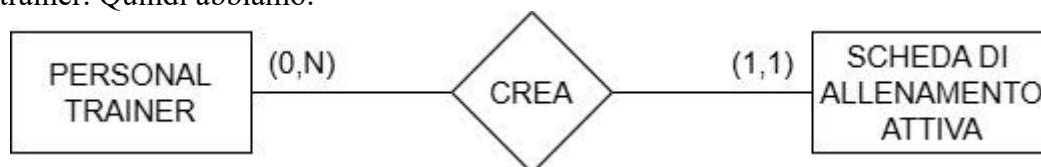
**Passo 8:** Cardinalità relazione “Contenuto”.

Dopo aver raffinato l’entità “Scheda di allenamento” con una generalizzazione, possiamo collegare l’entità “Esercizi” all’entità “Scheda di allenamento attiva” con la relazione “Contenuto”. L’entità “Esercizi” partecipa alla relazione “Contenuto” con cardinalità (0,N), poiché un esercizio si può trovare in più schede di allenamento, mentre l’entità “Scheda di allenamento attiva” partecipa alla relazione con cardinalità (1,N), dato che una scheda di allenamento può avere più esercizi. Dunque abbiamo:



Passo 9: Cardinalità relazione “Crea”.

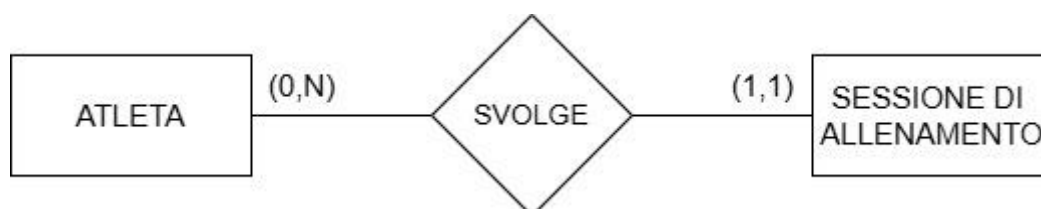
Dopo aver raffinato l'entità “Scheda di allenamento” con una generalizzazione, possiamo collegare l'entità “Personal trainer” all'entità “Scheda di allenamento attiva” con la relazione “Crea”. L'entità “Personal Trainer” partecipa alla relazione con cardinalità (0, N), dato che un personal trainer può creare più schede di allenamento, mentre l'entità “Scheda di allenamento attiva” partecipa alla relazione con cardinalità (1, 1), dato che una scheda di allenamento può essere creata da un solo personal trainer. Quindi abbiamo:

**Passo 10:** Inserimento relazione “Visualizza” e cardinalità.

Grazie alla generalizzazione che ci ha permesso di raffinare l'entità “Scheda di allenamento” e dopo aver letto le specifiche, si può aggiungere una relazione tra “Atleta” e “Scheda di allenamento archiviata”, denominata “Visualizza”. Questo perché ogni atleta può visualizzare le schede di allenamento archiviate, senza poter interagire con esse. Dunque abbiamo che l'entità “Atleta” partecipa alla relazione con cardinalità (0, N), dato che un atleta può visualizzare più schede di allenamento archiviate, ma può anche non visualizzarne nessuna, invece l'entità “Scheda di allenamento archiviata” partecipa alla relazione con cardinalità (0, 1), dato che una scheda di allenamento archiviata può appartenere ad un solo atleta. Per cui lo schema intermedio è:

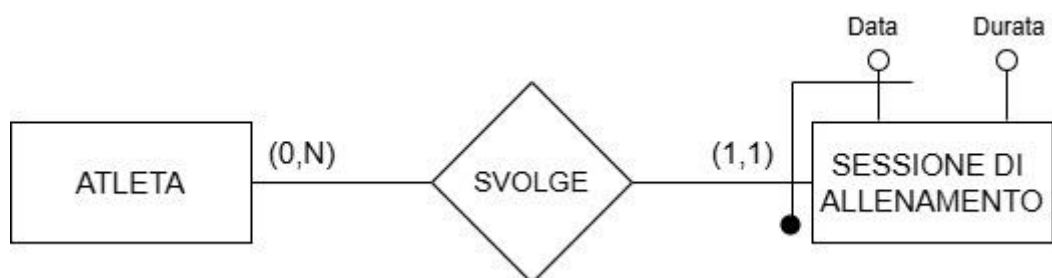
**Passo 11:** Cardinalità relazione “Svolge”.

L'entità “Atleta” partecipa alla relazione con cardinalità (0, N), questo perché un atleta può svolgere più sessioni di allenamento, mentre l'entità “Sessione di allenamento” partecipa alla relazione con cardinalità (1, 1), questo perché ad ogni sessione è associato un singolo atleta. Infatti abbiamo:



Passo 12: Raffinamento entità “Sessione di allenamento”.

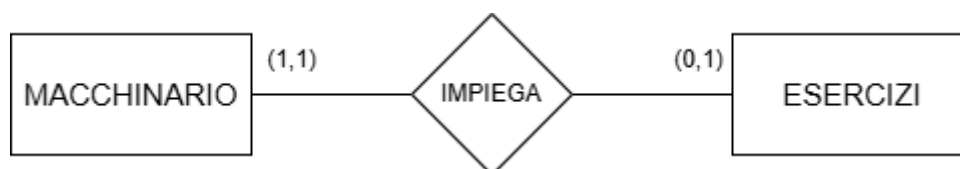
L’entità “Sessione di allenamento” ha i seguenti attributi: la data, che insieme al codice fiscale dell’atleta, rappresentano l’identificatore, e la durata della sessione. Per cui abbiamo:

**Passo 13:** Cardinalità relazione “Utilizza”.

L’entità “Scheda di allenamento attiva” partecipa alla relazione con cardinalità $(1, 1)$, perché una scheda di allenamento può essere associata ad un solo atleta, mentre l’entità “Atleta” partecipa alla relazione con cardinalità $(0, 1)$, perché un atleta può avere al massimo 1 scheda di allenamento attiva. Dunque:

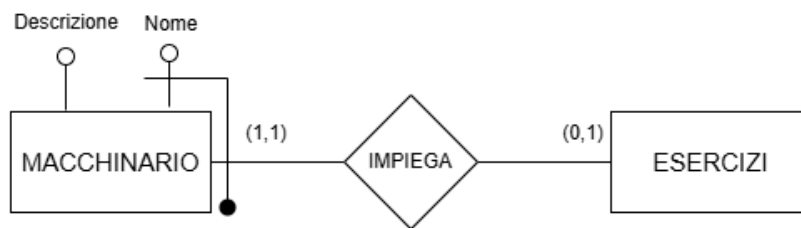
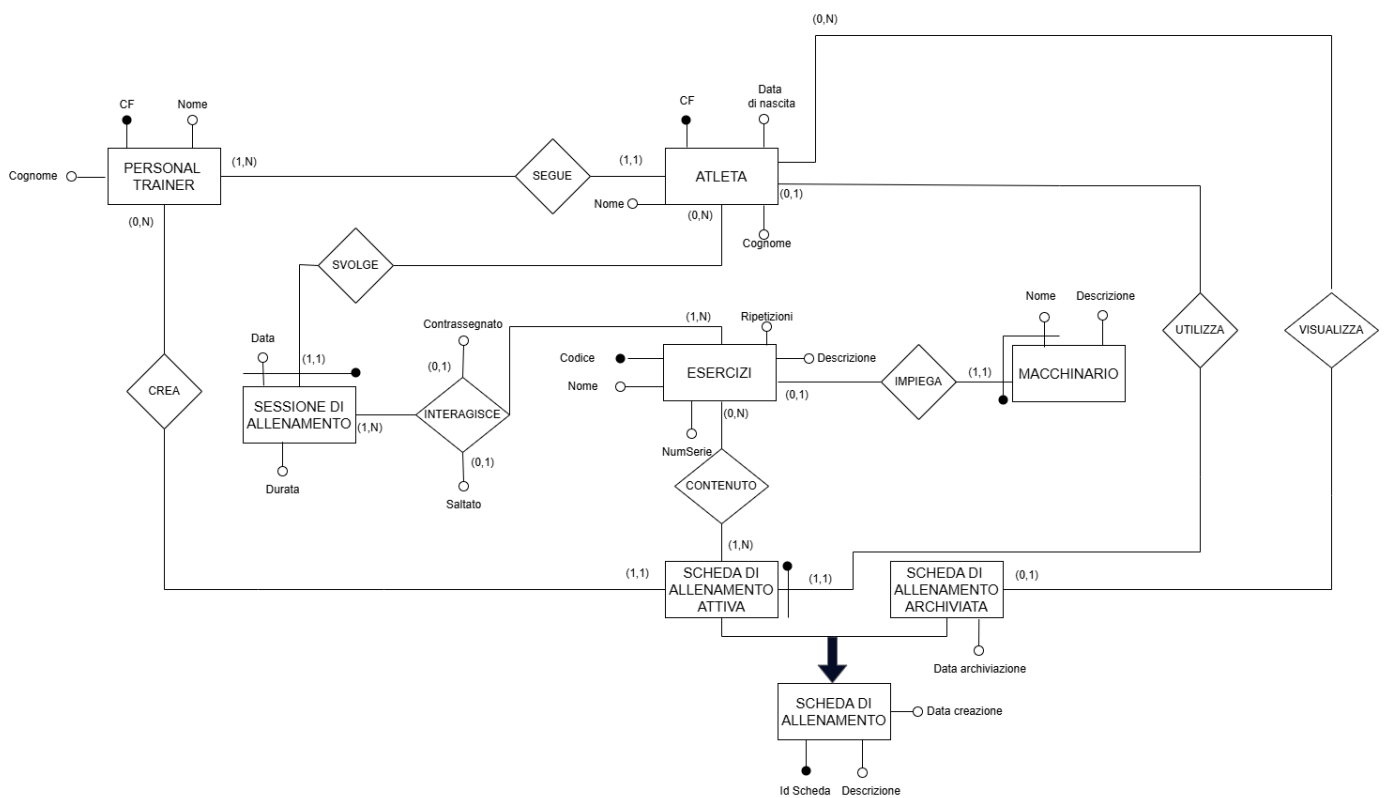
**Passo 14:** Cardinalità della relazione “Impiega”.

L’entità “Esercizi” partecipa alla relazione con cardinalità $(0, 1)$, dato che non tutti gli esercizi sfruttano l’utilizzo di un macchinario, mentre l’entità “Macchinario” partecipa alla relazione con cardinalità $(1, 1)$, dato che un macchinario deve essere utilizzato per forza durante un esercizio. Dunque:



Passo 15: Raffinamento entità “Macchinario”.

L’entità “Macchinario” ha i seguenti attributi: il nome, che insieme al codice dell’esercizio, rappresentano l’identificatore, e una descrizione. Per cui abbiamo:

**Integrazione finale**

Regole aziendali

- (RV1) Un atleta può avere una sola scheda di allenamento attiva alla volta.
- (RV2) Il gestore della palestra definisce un insieme di macchinari e di esercizi.
- (RV3) Una scheda di allenamento archiviata può solo essere visualizzata.
- (RV4) Un esercizio può essere presente in più schede di allenamento attive.
- (RV5) Un atleta può interagire solo con gli esercizi della propria scheda di allenamento attiva.
- (RV6) Il personal trainer può generare report solo per gli atleti che segue.
- (RV7) Il report può essere generato solo per le sessioni di allenamento già concluse.
- (RV8) Il report deve includere il numero di sessioni svolte da un atleta in un determinato intervallo temporale e la durata di ogni sessione.
- (RV9) Il report deve indicare il progresso (%) della scheda di allenamento utilizzata in ogni sessione di allenamento.
- (RV10) Il report deve includere il numero di esercizi completati ed il numero di esercizi saltati all'interno di ogni sessione di allenamento.

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
Personal trainer	Professionista che segue e crea schede di allenamento per gli atleti.	Nome, Cognome	CF
Atleta	Persona che esegue esercizi seguendo una scheda di allenamento durante una sessione di allenamento.	Nome, Cognome, Data di nascita	CF
Esercizi	Attività fisica che fa parte della scheda di allenamento.	Nome, Descrizione, NumSerie, Ripetizioni	Codice
Scheda di allenamento	Insieme di esercizi assegnati ad un atleta per il suo allenamento.	Descrizione, Data creazione	IdScheda
Scheda di allenamento attiva	Scheda attualmente in uso per l'atleta.	Descrizione, Data creazione	IdScheda, Atleta

Scheda di allenamento archiviata	Scheda non più in uso, ma salvata per riferimento e che può essere visualizzata dagli atleti.	Descrizione, Data creazione, Data archiviazione	IdScheda
Sessione di allenamento	Evento in cui un atleta svolge gli esercizi della scheda di allenamento.	Durata	Data, Atleta
Macchinario	Attrezzatura disponibile all'interno di una palestra, utilizzata per eseguire uno o più esercizi.	Descrizione	Esercizi, Nome

4. Progettazione logica

Volume dei dati

Si suppone che il sistema debba tenere traccia dei dati per 10 anni. Nel sistema operano circa 50 personal trainer, ognuno dei quali segue mediamente 5 atleti, per un totale di circa 250 atleti registrati.

Ogni atleta ha una scheda di allenamento attiva alla volta, che viene aggiornata ogni 6 mesi. Questo genera 250 nuove schede ogni semestre, per cui 500 nuove schede l'anno, per un totale di 5.000 schede in 10 anni, di cui circa 4.750 saranno archiviate. Supponiamo che ogni atleta svolga 2 sessioni di allenamento a settimana, per un totale di $2 \times 52 = 104$ sessioni l'anno per ogni atleta, per cui in 10 anni si registrano 260.000 sessioni di allenamento all'interno del sistema. Dato che ogni atleta, quando svolge una sessione, utilizza una scheda attiva, anche il volume della relazione "Utilizza" sarà 260.000.

Supponiamo che il catalogo degli esercizi disponibili sia composto da massimo 30 tipi di esercizi. Supponendo che ogni scheda contiene in media 4 esercizi, si ha che la relazione "Contenuto" avrà un volume atteso di circa 20.000. Ogni sessione include in media 4 esercizi, generando 1.000.000 di interazioni in 10 anni tra atleti e gli esercizi. Ogni atleta visualizza in media 1 scheda archiviata all'anno, per un totale di 2.500 visualizzazioni in 10 anni.

Inoltre supponiamo che il catalogo dei macchinari disponibili sia composto da 10 tipi di macchinari. Supponendo che in media 1 esercizio su 2 impiega un macchinario, si ha che la relazione "Impiega" avrà un volume atteso di circa 520.000 in 10 anni.

Infine supponiamo che gli esercizi svolti in 10 anni sono 1.300.000, se un atleta salta in media 1 esercizio ogni 2 sessioni, abbiamo 130.000 esercizi saltati in 10 anni, mentre gli esercizi svolti saranno 1.170.000.

Concetto nello schema	Tipo ¹	Volume atteso
Personal trainer	E	50
Atleta	E	250
Scheda di allenamento	E	5.000
Scheda di allenamento attiva	E	250
Scheda di allenamento archiviata	E	4.750
Sessione di allenamento	E	260.000
Esercizi	E	30
Macchinario	E	10

¹ Indicare con E le entità, con R le relazioni

Segue	R	2500
Svolge	R	260.000
Crea	R	5.000
Utilizza	R	260.000
Visualizza	R	2.500
Interagisce	R	1.000.000
Contenuto	R	20.000
Impiega	R	520.000

Tavola delle operazioni

Cod.	Descrizione	Frequenza attesa
OP1	Assegnazione di un personal trainer ad un atleta.	250/anno
OP2	Creazione di una nuova scheda di allenamento attiva.	10/settimana
OP3	Aggiornamento dell'elenco degli esercizi.	12/anno
OP4	Aggiornamento dell'elenco dei macchinari.	12/anno
OP5	Associazione di un esercizio ad una scheda di allenamento attiva.	40/settimana
OP6	Visualizzazione scheda di allenamento archiviata.	250/anno
OP7	Registrazione nuova sessione di allenamento.	70/giorno
OP8	Visualizzazione della scheda di allenamento attiva	70/giorno
OP9	Esercizio segnato come completato.	300/giorno
OP10	Esercizio segnato come saltato.	35/giorno
OP11	Generazione report.	10/settimana
OP12	Archiviazione di una scheda attiva.	10/settimana
OP13	Login	120/giorno

Costo delle operazioni

Supponiamo che un'operazione di scrittura costi il doppio rispetto ad un operazione in lettura, questo perché un accesso in lettura è meno costoso rispetto ad un accesso in scrittura. Consideriamo le seguenti operazioni:

- OP1 – Assegnazione di un personal trainer ad un atleta: in questo caso abbiamo 1 operazione di lettura sulle entità PERSONAL TRAINER e ATLETA, e 1 operazione di scrittura sull'entità SEGUE, dunque abbiamo che gli accessi totali sono: 1000 accessi/anno.
- OP2 - Creazione di una nuova scheda di allenamento attiva: in questo caso abbiamo 1 operazione in lettura su PERSONAL TRAINER, 1 operazione in scrittura sulla relazione

CREA e 1 operazione in scrittura sull'entità SCHEDA DI ALLENAMENTO ATTIVA, dunque abbiamo che gli accessi totali sono: 50 accessi/settimana.

- OP3 – Aggiornamento dell'elenco degli esercizi: in questo caso abbiamo 1 operazione scrittura sull'entità ESERCIZI, dunque gli accessi totali sono: 24 accessi/anno.
- OP4 – Aggiornamento dell'elenco dei macchinari: in questo caso abbiamo 1 operazione di scrittura sull'entità MACCHINARIO, dunque gli accessi totali sono: 24 accessi/anno.
- OP5 - Associazione di un esercizio ad una scheda di allenamento attiva: in questo caso abbiamo 1 operazione di lettura sull'entità ESERCIZI, 1 di lettura su SCHEDA DI ALLENAMENTO ATTIVA e 1 di scrittura sulla relazione CONTENUTO, dunque gli accessi totali sono: 160 accessi/settimana.
- OP6 - Visualizzazione scheda di allenamento archiviata: in questo caso abbiamo 1 operazione di lettura sulla relazione VISUALIZZA e una relazione di lettura sull'entità SCHEDA DI ALLENAMENTO ARCHIVIATA, dunque abbiamo che gli accessi totali sono: 500 accessi/anno.
- OP7 – Registrazione nuova sessione di allenamento: in questo caso abbiamo 1 operazione in lettura sull'entità ATLETA, 1 in scrittura sull'entità SVOLGE e 1 in scrittura sull'entità SESSIONE DI ALLENAMENTO, per cui abbiamo che gli accessi totali sono: 350 accessi/giorno.
- OP8 - Visualizzazione della scheda di allenamento attiva: in questo caso abbiamo 1 operazione in lettura sull'entità SCHEDA DI ALLENAMENTO ATTIVA, per cui abbiamo che gli accessi totali sono: 70 accessi/giorno.
- OP9 – Esercizio segnato come completato: in questo caso abbiamo 1 operazione in lettura sull'entità ATLETA e 1 sull'entità ESERCIZIO, mentre 1 operazione di scrittura sulla relazione INTERAGISCE, dunque gli accessi totali sono: 1200 accessi/giorno.
- OP10 – Esercizio segnato come saltato: in questo caso abbiamo 1 operazione in lettura sull'entità ATLETA e 1 sull'entità ESERCIZIO, mentre 1 operazione di scrittura sulla relazione INTERAGISCE, dunque gli accessi totali sono: 140 accessi/giorno.
- OP11 – Generazione report: in questo caso abbiamo 8 operazioni in lettura, ovvero su ATLETA, SESSIONE DI ALLENAMENTO, SCHEDA DI ALLENAMENTO, PERSONAL TRAINER, SVOLGE, UTILIZZA, INTERAGISCE e SEGUE, dunque gli accessi totali sono: 80 accessi/settimana. In questo caso non ci sono operazioni di scrittura, dato che una volta letti i dati, viene generato il report, ma non ci sono scritture permanenti sul database per la generazione di un report.

- OP12 – Archiviazione di una scheda attiva: in questo caso abbiamo solo 1 operazione di scrittura sull'entità SCHEDA DI ALLENAMENTO ARCHIVIATA, dunque gli accessi totali sono: 20 accessi/settimana.
- OP13 – Login: in questo caso gli accessi totali sono: 120 accessi/giorno, questo perché gli atleti accedono al sistema mediamente 2 volte a settimana (dato che le sessioni di allenamento settimanali in media sono 2), mentre i persona trainer accedono al sistema tutti i giorni.

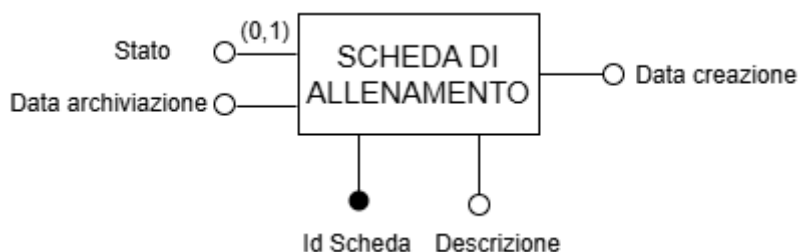
Ristrutturazione dello schema E-R

ANALISI DELLE RIDONDANZE:

Nella progettazione concettuale e logica effettuata in precedenza non si sono riscontrate ridondanze.

ELIMINAZIONE DELLE GENERALIZZAZIONI:

Le due entità figlie “Scheda di allenamento attiva” e “Scheda di allenamento archiviata”, possono essere accorpate nell'entità genitore “Scheda di allenamento”. Dobbiamo però introdurre un attributo booleano chiamato “Stato”, che sta ad indicare se la scheda è stata archiviata o meno. Così otteniamo:

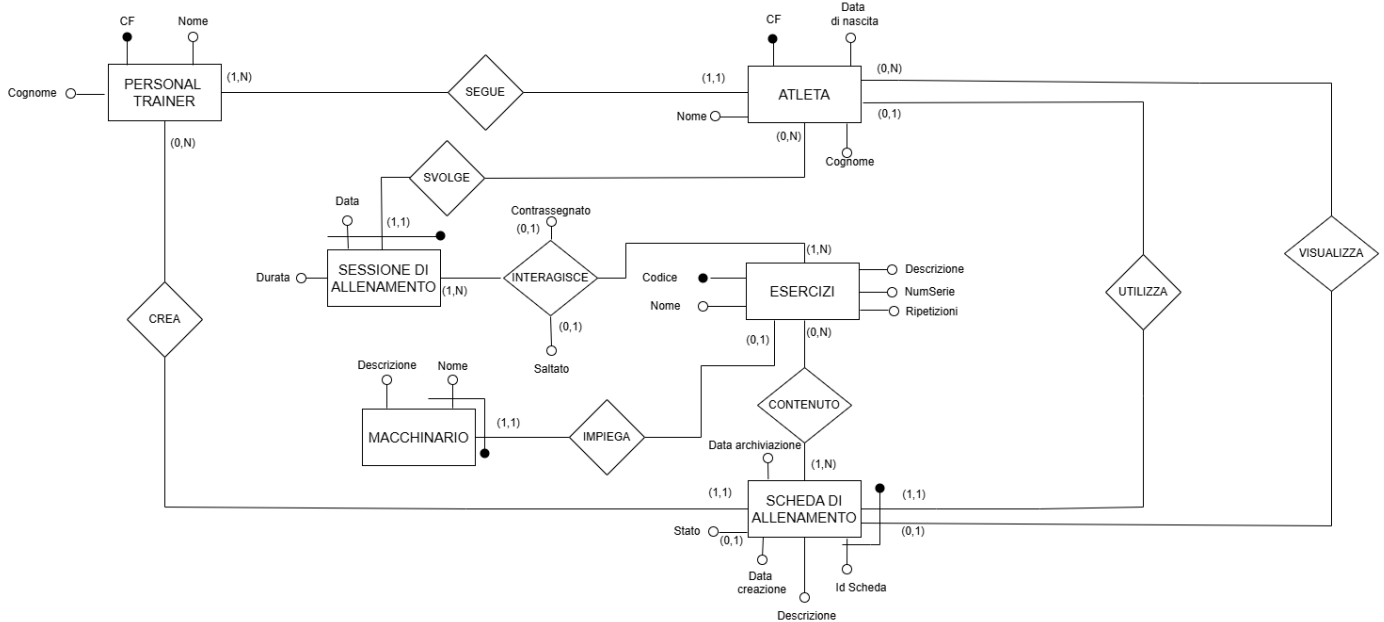


ELIMINAZIONE DEGLI ATRIBUTI MULTIVALORE:

Nello schema non sono presenti attributi multivalore.

SCELTA DEGLI IDENTIFICATORI PRIMARI:

Per gli identificatori primari si decide di lasciare quelli indicati nello schema concettuale. Pertanto lo schema ristrutturato è:



Trasformazione di attributi e identificatori

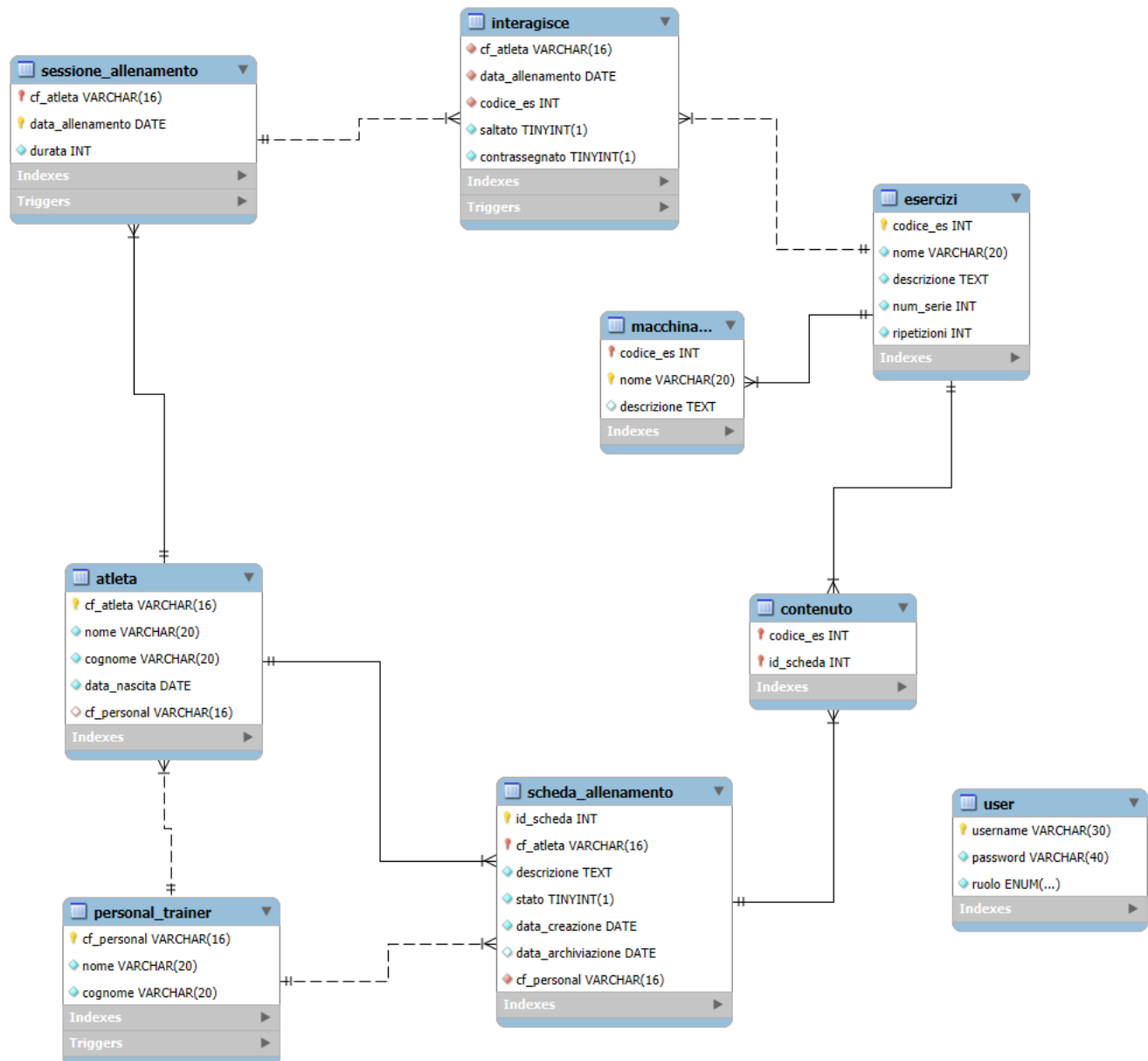
- Relazione “SEGUE”:
La relazione è identificata da “CF” di “PERSONAL TRAINER” e da “CF” di “ATLETA”. Il primo attributo viene ridenominato “cf_personal”, mentre il secondo “cf_atleta”.
- Entità “SESSIONE DI ALLENAMENTO”:
L’attributo “Data” di “SESSIONE DI ALLENAMENTO”, usato per rappresentare la relazione “SVOLGE” diventa “data_allenamento”.
- Entità “ESERCIZI”:
in questo caso per una maggiore chiarezza possiamo rinominare l’attributo “Codice” in “CodiceEs”.

Inoltre possiamo notare che nel nostro schema concettuale sono presenti 2 identificatori esterni: “CodiceEs” e “cf_atleta”. Il primo di questi viene utilizzato dall’entità MACCHINARIO perché ogni esercizio può dover impiegare un determinato macchinario per essere svolto.

Il secondo viene utilizzato da 2 entità: dall’entità SESSIONE DI ALLENAMENTO e dall’entità SCHEDA DI ALLENAMENTO, questo perché entrambe le entità devono essere associate ad un relativo atleta.

Traduzione di entità e associazioni

In questo paragrafo andiamo ad illustrare il modello relazionale completo dello schema concettuale di questa applicazione:



Normalizzazione del modello relazionale

Il modello relazionale precedentemente descritto è in forma normale di Boyce Codd (BCNF). Infatti, in tutte le dipendenze funzionali $X \rightarrow A$, X è chiave o superchiave. Analizziamo ogni caso:

- PersonalTrainer: abbiamo la seguente dipendenza funzionale:
 1. $cf_personal \rightarrow \text{Nome, Cognome}$per cui PersonalTrainer rispetta la BCNF, dato che $cf_personal$ è superchiave.
- Atleta: abbiamo la seguente dipendenza funzionale:
 1. $cf_atleta \rightarrow \text{Nome, Cognome, DataNascita, } cf_personal$per cui Atleta rispetta la BCNF, dato che cf_atleta è superchiave.
- SessioneDiAllenamento: abbiamo la seguente dipendenza funzionale:
 1. $\text{Atleta, data_allenamento} \rightarrow \text{Durata}$per cui SessioneDiAllenamento rispetta la BCNF, dato che Atleta e data_allenamento sono superchiavi.
- SchedaDiAllenamento: abbiamo la seguente dipendenza funzionale:
 1. $idScheda, cf_atleta \rightarrow \text{Descrizione, } cf_personal, \text{ Stato, DataCreazione, DataArchiviazione}$per cui SchedaDiAllenamento rispetta la BCNF, dato che idScheda e cf_atleta sono superchiavi.
- Esercizi: abbiamo la seguente dipendenza funzionale:
 1. $\text{CodiceEs} \rightarrow \text{Nome, Descrizione, NumSerie, Ripetizioni}$per cui Esercizi rispetta la BCNF, dato che CodiceEs è superchiave.
- Macchinario: abbiamo la seguente dipendenza funzionale:
 1. $\text{CodiceEs, Nome} \rightarrow \text{Descrizione}$per cui Macchinario rispetta la BCNF, dato che CodiceEs e Nome sono superchiavi.
- Contenuto: è in BCNF, infatti la superchiave non implica alcuna dipendenza funzionale non banale tra gli attributi.
- Interagisce: abbiamo la seguente dipendenza funzionale:
 1. $cf_atleta, data_allenamento, CodiceEs \rightarrow \text{Saltato, Contrassegnato}$per cui la relazione Interagisce rispetta la BCNF, dato che cf_atleta, data_allenamento e CodiceEs sono entrambi superchiavi.

5. Progettazione fisica

Utenti e privilegi

All'interno della base di dati abbiamo 4 tipi di utente: l'utente per effettuare il login, gli utenti Personal Trainer, gli utenti Atleta ed il gestore della palestra. In particolare:

- Alla prima tipologia sono assegnati i privilegi di EXECUTE per l'operazione OP13 (Login).
- Agli utenti Personal Trainer sono assegnati i privilegi di EXECUTE per le operazioni OP1 (Assegnazione di un personal trainer ad un atleta), OP2 (Creazione di una nuova scheda di allenamento attiva), OP5 (Associazione di un esercizio ad una scheda di allenamento attiva), OP11 (Generazione report) e OP12 (Archiviazione di una scheda attiva).
- Agli utenti Atleta sono assegnati i privilegi di EXECUTE per le operazioni OP6 (Visualizzazione scheda archiviata), OP7 (Registrazione nuova sessione di allenamento), OP8 (Visualizzazione della scheda di allenamento attiva), OP9 (Esercizio segnato come completato), OP10 (Esercizio segnato come saltato). Ha inoltre i privilegi SELECT sulla tabella "esercizi".
- Al gestore della palestra sono assegnati i privilegi di EXECUTE per l'operazione OP3 (Aggiornamento dell'elenco degli esercizi), OP4 (Aggiornamento dell'elenco dei macchinari). Ha inoltre i privilegi di INSERT sulle tabelle "atleta" e "personal_trainer" ed i privilegi di DELETE sulla tabelle "esercizi", "macchinario".

Strutture di memorizzazione

Tabella <personal_trainer>		
Colonna	Tipo di dato	Attributi ²
cf_personal	VARCHAR(16)	PK, NN, UQ
nome	VARCHAR(20)	NN
cognome	VARCHAR(20)	NN

Tabella <atleta>		
Colonna	Tipo di dato	Attributi
cf_atleta	VARCHAR(16)	PK, NN, UQ

² PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

nome	VARCHAR(20)	NN
cognome	VARCHAR(20)	NN
data_nascita	DATE	NN
cf_personal	VARCHAR(16)	NULL

Tabella <sessione allenamento>		
Colonna	Tipo di dato	Attributi
cf atleta	VARCHAR(16)	PK, NN
data allenamento	DATE	PK, NN
durata	INT	NN, UN

Tabella <scheda allenamento >		
Colonna	Tipo di dato	Attributi
id scheda	INT	PK, NN, UQ, AI
cf atleta	VARCHAR(16)	PK, NN
descrizione	TEXT	NN
stato	TINYINT	NN
data creazione	DATE	NN
data archiviazione	DATE	NULL
cf_personal	VARCHAR(16)	NN

Tabella <esercizi>		
Colonna	Tipo di dato	Attributi
codice_es	INT	PK, NN, UQ, AI
nome	VARCHAR(20)	NN
descrizione	TEXT	NN
num serie	INT	NN, UN
ripetizioni	INT	NN, UN

Tabella <macchinario>		
Colonna	Tipo di dato	Attributi
codice_es	INT	PK, NN
nome	VARCHAR(16)	PK, NN
descrizione	TEXT	NN

Tabella <contenuto>		
Colonna	Tipo di dato	Attributi
id scheda	INT	PK, NN
codice_es	INT	PK, NN

Tabella <interagisce>		
Colonna	Tipo di dato	Attributi
cf_atleta	VARCHAR(16)	PK, NN
dataAllenamento	DATE	PK, NN
codice_es	INT	PK, NN
saltato	TINYINT	NN
contrassegnato	TINYINT	NN

Tabella <user>		
Colonna	Tipo di dato	Attributi
username	VARCHAR(30)	PK, NN
password	VARCHAR(40)	NN
ruolo	ENUM('personal', 'atleta', 'gestore')	NN

Indici

Tabella <macchinario>	
Indice <macchinario_idx>	Tipo ³ :
Nome	<IDX>

L'indice è stato introdotto per migliorare le performance della query di eliminazione che filtra i macchinari per nome. Infatti, la procedura `elimina_macchinario` utilizza un'operazione `DELETE ... WHERE nome = ?`, che in modalità Safe Updates di MySQL richiede l'uso di una colonna indicizzata nella clausola `WHERE`. L'indice permette quindi sia di soddisfare il vincolo imposto da Safe Updates, sia di ridurre il costo computazionale della ricerca del record da eliminare, ottimizzando l'operazione.

³ IDX = index, UQ = unique, FT = full text, PR = primary.

Trigger

Il trigger “check_lunghezza_cf_personal” viene scatenato prima dell’inserimento di un personal trainer. Esso controlla che la lunghezza del codice fiscale sia esattamente uguale a 16, impedendo l’inserimento di codici fiscali non validi.

```
CREATE DEFINER='root'@'localhost' TRIGGER `check_lunghezza_cf_personal` BEFORE
INSERT ON `personal_trainer` FOR EACH ROW BEGIN

    IF CHAR_LENGTH(NEW.cf_personal) != 16 THEN

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Il codice fiscale del personal trainer deve essere lungo
esattamente 16 caratteri.';

    END IF;

END
```

Il trigger “check_lunghezza_cf_atleta” viene scatenato prima dell’inserimento di un atleta. Esso effettua lo stesso controllo del trigger visto in precedenza, ma riferito alla tabella “atleta”.

```
CREATE DEFINER='root'@'localhost' TRIGGER `check_lunghezza_cf_atleta` BEFORE INSERT
ON `atleta` FOR EACH ROW BEGIN

    IF CHAR_LENGTH(NEW.cf_atleta) != 16 THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Il codice fiscale dell\'atleta deve essere lungo esattamente 16 caratteri.';

    END IF;

END
```

Il trigger “max_5_esercizi_per_scheda” viene scatenato prima dell’inserimento di un nuovo esercizio all’interno della tabella contenuto. Esso controlla che il numero totale di esercizi associati a una stessa scheda non superi il limite massimo di 5, impedendo quindi l’inserimento di un sesto esercizio nella stessa scheda di allenamento.

```
CREATE DEFINER='root'@'localhost' TRIGGER `max_5_esercizi_per_scheda` BEFORE
INSERT ON `contenuto` FOR EACH ROW BEGIN

    DECLARE numero_esercizi INT;
```

```
SELECT COUNT(*) INTO numero_esercizi

FROM contenuto

WHERE id_scheda = NEW.id_scheda;

IF numero_esercizi >= 5 THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Non è possibile aggiungere più di 5 esercizi a una scheda';

END IF;

END
```

Il trigger “max_3_esercizi_saltati” viene scatenato prima dell’inserimento di una nuova interazione nella tabella interagisce. Esso verifica che, per una determinata sessione di allenamento (identificata da atleta e data), non siano già stati contrassegnati più di 3 esercizi come "saltati". Se l’inserimento comporterebbe il superamento di questo limite, l’operazione viene annullata, impedendo di saltare più di 3 esercizi in una singola sessione.

```
CREATE DEFINER='root'@'localhost' TRIGGER `max_3_esercizi_saltati` BEFORE INSERT ON
`interagisce` FOR EACH ROW BEGIN

    DECLARE esercizi_saltati INT;

    SELECT COUNT(*) INTO esercizi_saltati

    FROM interagisce

    WHERE cf_atleta = NEW.cf_atleta

    AND data_allenamento = NEW.data_allenamento

    AND saltato = 1;

    IF NEW.saltato = 1 THEN

        SET esercizi_saltati = esercizi_saltati + 1;

    END IF;

    IF esercizi_saltati > 3 THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Non è possibile saltare più di 3 esercizi per sessione';

    END IF;

END
```

```
END IF;
```

```
END
```

Il trigger “verifica_durata_sessione” viene scatenato prima dell’inserimento di una nuova sessione di allenamento. Esso controlla che la durata della sessione sia almeno di 30 minuti, impedendo l’inserimento se questo requisito non è rispettato.

```
CREATE DEFINER='root'@'localhost' TRIGGER `verifica_durata_sessione` BEFORE INSERT ON
```

```
`sessione_allenamento` FOR EACH ROW
```

```
BEGIN
```

```
IF NEW.durata < 30 THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'La durata minima di una sessione è di 30 minuti';
```

```
END IF;
```

```
END
```

Il trigger “check_singola_interazione” viene attivato prima dell’inserimento di una nuova interazione tra un atleta e un esercizio. Esso verifica due condizioni: la prima impedisce che un esercizio venga contemporaneamente contrassegnato come completato e saltato nella stessa registrazione; la seconda assicura che un esercizio già contrassegnato come completato per una determinata sessione non possa successivamente essere segnato come saltato. In questo modo si garantisce la coerenza dei dati relativi al completamento degli esercizi.

```
CREATE DEFINER='root'@'localhost' TRIGGER `check_singola_interazione` BEFORE INSERT
```

```
ON `interagisce` FOR EACH ROW BEGIN
```

```
IF NEW.contrassegnato = 1 AND NEW.saltato = 1 THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Un esercizio non può essere completato e saltato insieme';
```

```
END IF;
```

```
IF NEW.saltato = 1 THEN
```

```
    IF EXISTS (
```

```
        SELECT 1
```

```
        FROM interagisce
```

```
WHERE cf_atleta = NEW.cf_atleta

AND data_allenamento = NEW.data_allenamento

AND codice_es = NEW.codice_es

AND contrassegnato = 1

) THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Non è possibile segnare come saltato un esercizio già completato';

END IF;

END IF;

END
```

Eventi

L'evento istanziato viene utilizzato per rimuovere le schede di allenamento che sono state archiviate per più di 5 anni. Prima della loro istanziazione viene attivato lo scheduler DBMS tramite il comando:

```
SET GLOBAL event_scheduler = on;
```

Il codice “elimina_schede_archivate_vecchie” è il seguente:

```
CREATE EVENT IF NOT EXISTS elimina_schede_archivate_vecchie
```

```
ON SCHEDULE EVERY 1 DAY
```

```
STARTS CURRENT_TIMESTAMP
```

```
DO
```

```
DELETE FROM personal_trainer_digitale.scheda_allenamento
```

```
WHERE stato = 0
```

```
AND data_archiviazione IS NOT NULL
```

```
AND data_archiviazione < (CURRENT_DATE - INTERVAL 5 YEAR);
```

Non vengono utilizzati eventi effimeri.

Viste

La vista `dettagli_esercizi` serve a fornire una rappresentazione combinata di informazioni sugli esercizi e, se disponibile, sui macchinari associati. Dunque:

```
CREATE
  ALGORITHM = UNDEFINED
  DEFINER = `root`@`localhost`
  SQL SECURITY DEFINER
VIEW `dettagli_esercizi` AS
  SELECT
    `e`.`codice_es` AS `codice_es`,
    `e`.`nome` AS `nome_esercizio`,
    `e`.`descrizione` AS `descrizione_esercizio`,
    `e`.`num_serie` AS `num_serie`,
    `e`.`ripetizioni` AS `ripetizioni`,
    `m`.`nome` AS `nome_macchinario`,
    `m`.`descrizione` AS `descrizione_macchinario`
  FROM
    (`esercizi` `e`
    LEFT JOIN `macchinario` `m` ON ((`e`.`codice_es` = `m`.`codice_es`)))
```

La vista `report_allenamenti` fornisce un resoconto dettagliati delle sessioni di allenamento effettuate da ciascun atleta, con informazioni sull'andamento delle schede di allenamento associate ai vari allenamenti. Dunque:

```
CREATE
  ALGORITHM = UNDEFINED
  DEFINER = `root`@`localhost`
  SQL SECURITY DEFINER
VIEW `report_allenamenti` AS
  SELECT
    `a`.`cf_atleta` AS `cf_atleta`,
    CONCAT(`a`.`nome`, ' ', `a`.`cognome`) AS `nome_atleta`,
    `sa`.`data_allenamento` AS `data_allenamento`,
```

```

`sa`.`durata` AS `durata`,
IFNULL(`sa_scheda`.`id_scheda`,
      'Nessuna scheda') AS `id_scheda`,
IFNULL(`sa_scheda`.`descrizione`,
      'Nessuna scheda') AS `descrizione_scheda`,
IFNULL(`sa_scheda`.`esercizi_assegnati`, 0) AS `esercizi_assegnati`,
COUNT(`i`.`codice_es`) AS `esercizi_eseguiti`,
SUM((CASE
      WHEN (`i`.`saltato` = 0) THEN 1
      ELSE 0
END)) AS `esercizi_completati`,
SUM((CASE
      WHEN (`i`.`saltato` = 1) THEN 1
      ELSE 0
END)) AS `esercizi_saltati`,
SUM((CASE
      WHEN
        ((`i`.`saltato` = 0)
         OR (`i`.`saltato` = 1))
      THEN
        1
      ELSE 0
END)) AS `totale_esercizi`,
ROUND(IFNULL(((SUM((CASE
      WHEN (`i`.`saltato` = 0) THEN 1
      ELSE 0
END)) * 100.0) / NULLIF(COUNT(`i`.`codice_es`), 0)),
      2),
      0) AS `percentuale_completamento`,
`a`.`cf_personal` AS `cf_personal`
FROM
  (((`atleta` `a`
JOIN `sessione_allenamento` `sa` ON ((`a`.`cf_atleta` = `sa`.`cf_atleta`)))
JOIN `interagisce` `i` ON (((`i`.`cf_atleta` = `sa`.`cf_atleta`))

```

```

AND ('i`.`data_allenamento` = `sa`.`data_allenamento`))))
LEFT JOIN (SELECT
`s1`.`id_scheda` AS `id_scheda`,
`s1`.`cf_atleta` AS `cf_atleta`,
`s1`.`descrizione` AS `descrizione`,
`s1`.`data_creazione` AS `data_creazione`,
(SELECT
COUNT(0)
FROM
`contenuto` `c`
WHERE
(`c`.`id_scheda` = `s1`.`id_scheda`)) AS `esercizi_assegnati`
FROM
`scheda_allenamento` `s1`) `sa_scheda` ON (((`sa_scheda`.`cf_atleta` = `a`.`cf_atleta`)
AND (`sa_scheda`.`data_creazione` = (SELECT
MAX(`s2`.`data_creazione`)
FROM
`scheda_allenamento` `s2`
WHERE
((`s2`.`cf_atleta` = `a`.`cf_atleta`)
AND (`s2`.`data_creazione` <= `sa`.`data_allenamento`))))))
GROUP BY `a`.`cf_atleta` , `a`.`nome` , `a`.`cognome` , `sa`.`data_allenamento` , `sa`.`durata` ,
`sa_scheda`.`id_scheda` , `sa_scheda`.`descrizione` , `sa_scheda`.`esercizi_assegnati` ,
`a`.`cf_personal`

```

Stored Procedures e transazioni

OP1

Questa procedura assegna un Personal Trainer a un Atleta, aggiornando il riferimento al codice fiscale del personal trainer nella tabella degli atleti. La procedura effettua controlli di esistenza su entrambi i soggetti prima di eseguire l'aggiornamento. Viene gestita come transazione atomica, con un handler di errore che esegue un rollback in caso di eccezioni.

Il livello di isolamento della transazione è impostato su REPEATABLE READ, per garantire che i dati letti durante la transazione restino coerenti fino al commit, prevenendo letture non ripetibili.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `assegna_personal`(in var_cf_atleta VARCHAR(16),  
in var_cf_personal VARCHAR(16))
```

```
BEGIN
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
    BEGIN
```

```
        ROLLBACK;
```

```
        RESIGNAL;
```

```
    END;
```

```
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
    START TRANSACTION;
```

```
    IF NOT EXISTS (SELECT 1 FROM personal_trainer_digitale.personal_trainer WHERE cf_personal =  
var_cf_personal) THEN
```

```
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Personal Trainer non trovato';
```

```
    END IF;
```

```
    IF NOT EXISTS (SELECT 1 FROM personal_trainer_digitale.atleta WHERE cf_atleta = var_cf_atleta)  
    THEN
```

```
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Atleta non trovato';
```

```
    END IF;
```

```
    UPDATE `personal_trainer_digitale`.`atleta`
```

```
    SET `cf_personal` = var_cf_personal
```

```
    WHERE `cf_atleta` = var_cf_atleta;
```

COMMIT;

END

OP2

Questa procedura consente a un Personal Trainer di creare una nuova scheda di allenamento attiva per un determinato Atleta, gestendo in modo automatico l'eventuale presenza di una scheda attiva precedente; infatti prima del nuovo inserimento, se l'atleta possiede già una scheda attiva, l'attributo "stato" di tale scheda viene posto a 0, così da far capire che la scheda è archiviata.

Viene utilizzato il livello di isolamento REPEATABLE READ, poiché durante l'intera transazione è necessario mantenere coerenza tra le letture, evitando fenomeni come phantom reads. Questo livello di isolamento è particolarmente adatto nel contesto della creazione di schede di allenamento, in quanto previene inserimenti o modifiche concorrenti che potrebbero portare alla creazione simultanea di più schede attive per uno stesso atleta.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `creazione_scheda_attiva`(in var_cf_personal  
VARCHAR(16), in var_cf_atleta VARCHAR(16), in var_descrizione TEXT)
```

```
BEGIN
```

```
    DECLARE id_vecchia_scheda INT;
```

```
    DECLARE descrizione_vecchia TEXT;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
    BEGIN
```

```
        ROLLBACK;
```

```
        RESIGNAL;
```

```
    END;
```

```
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
    START TRANSACTION;
```

```
    IF NOT EXISTS (
```

```
        SELECT 1
```

```
        FROM personal_trainer_digitale.personal_trainer
```

```
        WHERE cf_personal = var_cf_personal
```

```
    ) THEN
```

```
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Personal Trainer non trovato';
```

```
    END IF;
```

```
    SELECT id_scheda, descrizione
```

```
INTO id_vecchia_scheda, descrizione_vecchia

FROM personal_trainer_digitale.scheda_allenamento

WHERE cf_atleta = var_cf_atleta AND stato = 1

LIMIT 1;

IF id_vecchia_scheda IS NOT NULL THEN

    UPDATE personal_trainer_digitale.scheda_allenamento

    SET stato = 0,
        data_archiviazione = CURDATE()

    WHERE id_scheda = id_vecchia_scheda;

END IF;

INSERT INTO personal_trainer_digitale.scheda_allenamento (
    descrizione,
    stato,
    data_archiviazione,
    data_creazione,
    cf_atleta,
    cf_personal
)

VALUES (
    var_descrizione,
    1,
    NULL,
    CURDATE(),
    var_cf_atleta,
    var_cf_personal
);

COMMIT;

END
```

OP3

Questa procedura consente di aggiungere un nuovo esercizio al database, specificandone nome, descrizione, numero di serie e ripetizioni. La transazione è gestita al livello di isolamento READ COMMITTED perché l'operazione è una semplice scrittura, e non necessita di visibilità stabile dei dati letti. L'operazione è garantita da una transazione esplicita che mantiene la coerenza dei dati.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `aggiorna_esercizi`(in var_nome
VARCHAR(20), in var_descrizione TEXT, in var_num_serie INT UNSIGNED, in var_ripetizioni
INT UNSIGNED)
```

```
BEGIN
```

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

    ROLLBACK;

    RESIGNAL;

END;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

INSERT INTO esercizi (
    nome, descrizione, num_serie, ripetizioni
) VALUES (
    var_nome, var_descrizione, var_num_serie, var_ripetizioni
);

COMMIT;

END
```

OP4

Questa procedura permette di aggiungere un nuovo macchinario al sistema, associandolo a un determinato esercizio (tramite il codice_es) e fornendo una descrizione e un nome del macchinario. Il livello di isolamento della transazione è impostato su READ COMMITTED, adatto a operazioni di scrittura che non richiedono visibilità stabile dei dati letti. L'inserimento di un macchinario è una semplice operazione di scrittura nel database.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `aggiorna_macchinari`(in var_codice_es
INT, in var_nome VARCHAR(20), in var_descrizione TEXT)
```

```
BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

    ROLLBACK;

    RESIGNAL;

END;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;
```

```
INSERT INTO `personal_trainer_digitale`.`macchinario` (  
    `codice_es`,  
    `nome`,  
    `descrizione`  
) VALUES (  
    var_codice_es,  
    var_nome,  
    var_descrizione  
);  
  
COMMIT;
```

END

OP5

Questa procedura consente di associare un esercizio, identificato dal suo codice (codice_es), a una specifica scheda di allenamento (id_scheda). Prima di effettuare l'associazione, vengono eseguiti controlli per verificare l'esistenza dell'esercizio e la validità della scheda: in particolare, l'associazione è consentita solo se la scheda è attiva (cioè con stato pari a 1). Il livello di isolamento è impostato su REPEATABLE READ, per garantire coerenza tra le letture durante la transazione e prevenire letture non ripetibili.

Questa procedura fa scattare il trigger “max_5_esercizi_per_scheda”.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `associa_esercizio_scheda`(IN  
var_codice_es INT, IN var_id_scheda INT)
```

```
BEGIN
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    RESIGNAL;
```

```
END;
```

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
START TRANSACTION;
```

```
IF NOT EXISTS (  
    SELECT 1
```

```
    FROM `personal_trainer_digitale`.`esercizi`
```

```
    WHERE `codice_es` = var_codice_es
```

```
) THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Esercizio non trovato';
```

```
END IF;

IF NOT EXISTS (
    SELECT 1
    FROM `personal_trainer_digitale`.`scheda_allenamento`
    WHERE `id_scheda` = var_id_scheda
    AND `stato` = 1
) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Scheda di allenamento non attiva o non trovata';

END IF;

INSERT INTO `personal_trainer_digitale`.`contenuto` (`codice_es`, `id_scheda`)

VALUES (var_codice_es, var_id_scheda);

COMMIT;

END
```

OP6

Questa procedura consente di visualizzare tutte le schede di allenamento archiviate associate a un atleta, identificato dal suo codice fiscale (cf_atleta), insieme agli esercizi collegati a ciascuna scheda. Prima di procedere, vengono effettuati controlli per verificare l'esistenza dell'atleta e per assicurarsi che abbia almeno una scheda archiviata. La transazione è read-only e utilizza il livello di isolamento READ COMMITTED, che evita letture sporche, garantendo al contempo performance efficienti durante le operazioni di lettura. La procedura restituisce informazioni sulle schede archiviate, insieme ai dettagli degli esercizi associati, ordinati per ID della scheda e nome dell'esercizio.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `visualizza_scheda_archiviata`(in
var_cf_atleta VARCHAR(16))
```

```
BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

    ROLLBACK;

    RESIGNAL;

END;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SET TRANSACTION READ ONLY;

START TRANSACTION;
```

```
IF NOT EXISTS (  
    SELECT 1  
    FROM personal_trainer_digitale.atleta  
    WHERE cf_atleta = var_cf_atleta  
) THEN  
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Atleta non trovato';  
  
END IF;  
  
IF NOT EXISTS (  
    SELECT 1  
    FROM personal_trainer_digitale.scheda_allenamento  
    WHERE cf_atleta = var_cf_atleta AND stato = 0  
) THEN  
  
    SIGNAL SQLSTATE '45000'  
  
    SET MESSAGE_TEXT = 'Nessuna scheda archiviata per questo atleta';  
  
END IF;  
  
SELECT  
    sa.id_scheda,  
    sa.descrizione AS descrizione_scheda,  
    sa.data_archiviazione,  
    e.codice_es,  
    e.nome AS nome_esercizio,  
    e.descrizione AS descrizione_esercizio,  
    e.num_serie,  
    e.ripetizioni  
  
FROM  
    personal_trainer_digitale.scheda_allenamento sa  
  
LEFT JOIN  
    personal_trainer_digitale.contenuto c ON sa.id_scheda = c.id_scheda  
  
LEFT JOIN  
    personal_trainer_digitale.esercizi e ON c.codice_es = e.codice_es  
  
WHERE  
    sa.cf_atleta = var_cf_atleta  
    AND sa.stato = 0  
  
ORDER BY  
    sa.id_scheda, e.nome;  
  
COMMIT;  
END
```

OP7

Questa procedura memorizza una nuova sessione di allenamento per un atleta, includendo il codice fiscale dell'atleta, la data dell'allenamento e la durata in minuti.

La transazione utilizza il livello di isolamento REPEATABLE READ per garantire la coerenza dei dati durante la verifica dell'esistenza dell'atleta e l'inserimento della sessione, evitando letture inconsistenti o modifiche concorrenti.

Inoltre questa procedura fa scattare il trigger “verifica_durata_sessione”.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `registrazione_allenamento`(in  
var_cf_atleta VARCHAR(16), in var_data_allenamento DATE, in var_durata INT UNSIGNED)
```

```
BEGIN
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    RESIGNAL;
```

```
END;
```

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
START TRANSACTION;
```

```
IF NOT EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`atleta` WHERE `cf_atleta` =  
var_cf_atleta) THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Atleta non trovato';
```

```
END IF;
```

```
INSERT INTO `personal_trainer_digitale`.`sessione_allenamento`  
    (`cf_atleta`, `data_allenamento`, `durata`)
```

```
VALUES
```

```
    (var_cf_atleta, var_data_allenamento, var_durata);
```

```
    COMMIT;
```

```
END
```

OP8

Questa procedura permette di visualizzare la scheda attiva associata a un determinato atleta. Viene eseguito un controllo di esistenza sull'atleta specificato e sulla presenza di una scheda attiva associata al suo personal trainer.

Si utilizza il livello di isolamento READ COMMITTED, in quanto l'operazione è di sola lettura e deve garantire che i dati letti siano committed. Viene specificato anche READ ONLY per ottimizzare l'esecuzione della transazione in modalità esclusivamente consultiva.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `visualizza_scheda_attiva`(in var_cf_atleta  
VARCHAR(16))
```

```
BEGIN
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    RESIGNAL;
```

```
END;
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
SET TRANSACTION READ ONLY;
```

```
START TRANSACTION;
```

```
IF NOT EXISTS (SELECT 1 FROM personal_trainer_digitale.atleta WHERE cf_atleta =  
var_cf_atleta) THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Atleta non trovato';
```

```
END IF;
```

```
IF NOT EXISTS (
```

```
    SELECT 1
```

```
    FROM personal_trainer_digitale.scheda_allenamento
```

```
    WHERE cf_atleta = var_cf_atleta AND stato = 1
```

```
) THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Nessuna scheda attiva trovata per  
l'atleta';
```

```
END IF;
```

```
SELECT
```

```
    sa.id_scheda,
```

```
    sa.descrizione AS descrizione_scheda,
```

```
    sa.cf_personal,
```

```
    e.codice_es,
```

```
    e.nome AS nome_esercizio,
```

```
e.num_serie,  
e.ripetizioni  
  
FROM  
    personal_trainer_digitale.scheda_allenamento sa  
  
INNER JOIN  
    personal_trainer_digitale.contenuto c ON sa.id_scheda = c.id_scheda  
  
INNER JOIN  
    personal_trainer_digitale.esercizi e ON c.codice_es = e.codice_es  
  
WHERE  
    sa.cf_atleta = var_cf_atleta  
    AND sa.stato = 1  
  
ORDER BY  
    sa.id_scheda, e.nome;  
  
COMMIT;  
  
END
```

OP9

Questa procedura registra il completamento di un esercizio da parte di un atleta in una specifica data di allenamento, assicurando che l'esercizio faccia parte della scheda attiva dell'atleta alla data dell'allenamento. Il livello di isolamento è impostato su REPEATABLE READ per garantire la coerenza delle letture all'interno della transazione. Inoltre, questa procedura fa scattare il trigger “check_singola_interazione”.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `esercizio_completato`(in var_cf_atleta  
VARCHAR(16), in var_codice_es INT, in var_data_allenamento DATE)  
  
BEGIN  
  
    DECLARE EXIT HANDLER FOR SQLEXCEPTION  
  
BEGIN  
  
    ROLLBACK;  
  
    RESIGNAL;  
  
END;  
  
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
  
START TRANSACTION;
```

```
IF NOT EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`sessione_allenamento` WHERE  
`cf_atleta` = var_cf_atleta) THEN
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Atleta non trovato';
```

```
END IF;
```

```
IF NOT EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`esercizi` WHERE  
`codice_es` = var_codice_es) THEN
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Esercizio non trovato';
```

```
END IF;
```

```
IF NOT EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`sessione_allenamento` WHERE  
`data_allenamento` = var_data_allenamento) THEN
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Allenamento non trovato in quella  
data';
```

```
END IF;
```

```
IF NOT EXISTS (  
SELECT 1
```

```
FROM scheda_allenamento sa
```

```
JOIN contenuto c ON sa.id_scheda = c.id_scheda
```

```
WHERE sa.cf_atleta = var_cf_atleta
```

```
AND sa.data_creazione = (  
SELECT MAX(s2.data_creazione)
```

```
FROM scheda_allenamento s2
```

```
WHERE s2.cf_atleta = var_cf_atleta
```

```
AND s2.data_creazione <= var_data_allenamento
```

```
)
```

```
AND c.codice_es = var_codice_es
```

```
) THEN
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'L'esercizio non fa parte  
della scheda attiva dell'atleta alla data dell'allenamento';
```

```
END IF;
```

```
IF EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`interagisce`
```

```
WHERE `cf_atleta` = var_cf_atleta AND `data_allenamento` = var_data_allenamento  
AND `codice_es` = var_codice_es AND `contrassegnato` = 1) THEN
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'L'esercizio è già stato completato';

END IF;

IF NOT EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`interagisce`

    WHERE `cf_atleta` = var_cf_atleta AND `data_allenamento` = var_data_allenamento
    AND `codice_es` = var_codice_es) THEN

    INSERT INTO `personal_trainer_digitale`.`interagisce` (cf_atleta, data_allenamento, codice_es,
    saltato, contrassegnato)

        VALUES (var_cf_atleta, var_data_allenamento, var_codice_es, 0, 1);

ELSE

    UPDATE `personal_trainer_digitale`.`interagisce`

        SET `contrassegnato` = 1

        WHERE `cf_atleta` = var_cf_atleta AND `data_allenamento` = var_data_allenamento
        AND `codice_es` = var_codice_es;

END IF;

COMMIT;

END
```

OP10

La procedura esercizio_saltato consente di registrare che un determinato esercizio è stato saltato da un atleta durante una specifica sessione di allenamento. Effettua una serie di controlli per garantire la coerenza e la validità dei dati prima di aggiornare o inserire un record nella tabella interagisce. Il livello di isolamento della transazione è REPEATABLE READ, per garantire la coerenza delle letture durante la transazione. Questa procedura fa scattare il trigger “check_singola_interazione” ed il trigger “max_3_esercizi_saltati”.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `esercizio_saltato`(in var_cf_atleta
VARCHAR(16), in var_codice_es INT, in var_data_allenamento DATE)

BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

    ROLLBACK;
```

```
RESIGNAL;  
  
END;  
  
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
  
START TRANSACTION;  
  
IF NOT EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`sessione_allenamento` WHERE  
`cf_atleta` = var_cf_atleta) THEN  
  
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Atleta non trovato';  
  
END IF;  
  
IF NOT EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`esercizi` WHERE `codice_es` =  
var_codice_es) THEN  
  
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Esercizio non trovato';  
  
END IF;  
  
IF NOT EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`sessione_allenamento` WHERE  
`data_allenamento` = var_data_allenamento) THEN  
  
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Sessione non trovata per quella data';  
  
END IF;  
  
IF NOT EXISTS (  
    SELECT 1  
    FROM scheda_allenamento sa  
    JOIN contenuto c ON sa.id_scheda = c.id_scheda  
    WHERE sa.cf_atleta = var_cf_atleta  
    AND sa.data_creazione = (  
        SELECT MAX(s2.data_creazione)  
        FROM scheda_allenamento s2  
  
        WHERE s2.cf_atleta = var_cf_atleta  
        AND s2.data_creazione <= var_data_allenamento  
    )  
    AND c.codice_es = var_codice_es  
) THEN  
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'L'esercizio non fa parte  
della scheda attiva dell'atleta alla data dell'allenamento';  
  
END IF;  
  
IF EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`interagisce`
```

```

WHERE `cf_atleta` = var_cf_atleta AND `data_allenamento` = var_data_allenamento
AND `codice_es` = var_codice_es AND `saltato` = 1) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'L'esercizio è già stato saltato';

END IF;

IF EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`interagisce`

    WHERE `cf_atleta` = var_cf_atleta AND `data_allenamento` = var_data_allenamento AND
`codice_es` = var_codice_es AND `contrassegnato` = 1) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'L'esercizio è già stato completato';

END IF;

IF NOT EXISTS (SELECT 1 FROM `personal_trainer_digitale`.`interagisce`

    WHERE `cf_atleta` = var_cf_atleta AND `data_allenamento` = var_data_allenamento
AND `codice_es` = var_codice_es) THEN

    INSERT INTO `personal_trainer_digitale`.`interagisce` (cf_atleta, data_allenamento, codice_es,
saltato, contrassegnato)
    VALUES (var_cf_atleta, var_data_allenamento, var_codice_es, 1, 0);

ELSE

    UPDATE `personal_trainer_digitale`.`interagisce`

    SET `saltato` = 1

    WHERE `cf_atleta` = var_cf_atleta AND `data_allenamento` AND `codice_es` =
var_codice_es;

END IF;

COMMIT;

END

```

OP11

La procedura genera_report consente di ottenere un riepilogo degli allenamenti svolti dagli atleti seguiti da un determinato personal trainer, in un intervallo di date specificato. Prima di eseguire la query, vengono effettuati controlli di validità sui parametri in ingresso: si verifica l'esistenza del personal trainer e che la data di inizio non sia successiva alla data di fine. Viene impostato il livello di isolamento READ COMMITTED per garantire l'accesso a dati già confermati e si utilizza una transazione per assicurare coerenza in caso di errore. I dati restituiti provengono dalla vista report_allenamenti, filtrati in base al codice fiscale del personal trainer e al periodo selezionato.

```

CREATE DEFINER='root'@'localhost' PROCEDURE `genera_report`(in var_data_inizio DATE,
in var_data_fine DATE, in var_cf_personal VARCHAR(16))

```

```
BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

IF NOT EXISTS (
    SELECT 1 FROM personal_trainer WHERE cf_personal = var_cf_personal
) THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Personal trainer non trovato';

END IF;

IF var_data_inizio > var_data_fine THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Data inizio maggiore di data fine';

END IF;

SELECT *

FROM report_allenamenti

WHERE data_allenamento BETWEEN var_data_inizio AND var_data_fine
    AND cf_personal = var_cf_personal;

COMMIT;

END
```

OP12

La procedura `archivia_scheda_attiva` consente di archiviare una scheda di allenamento attiva impostando il suo stato a 0 e registrando la data di archiviazione. In questo modo, la scheda non viene eliminata ma resa non più attiva, permettendo di conservarne la cronologia per consultazioni

future.

Viene utilizzato il livello di isolamento READ COMMITTED, adeguato per evitare letture sporche durante l'operazione.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `archivia_scheda_attiva`(IN var_id_scheda INT)
```

```
BEGIN
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    RESIGNAL;
```

```
END;
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION;
```

```
UPDATE scheda_allenamento
```

```
SET stato = 0, -- 0 = archiviata  
    data_archiviazione = CURDATE()
```

```
WHERE id_scheda = var_id_scheda;
```

```
COMMIT;
```

```
END
```

OP13

Questa procedure permette agli utenti di effettuare il login, in modo tale da avere gli accessi alle operazioni che devono eseguire.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `login`(in var_username VARCHAR(30),  
in var_password VARCHAR(20), out ruolo_out INT)
```

```
BEGIN
```

```
    DECLARE var_ruolo ENUM('personal', 'atleta', 'gestore');
```

```
    SELECT `ruolo`
```

```
into var_ruolo
```

```
FROM `user`

WHERE `username` = var_username and `password` = var_password;

IF var_ruolo = 'personal' THEN

    SET ruolo_out = 1;

ELSEIF var_ruolo = 'atleta' THEN

    SET ruolo_out = 2;

ELSEIF var_ruolo = 'gestore' THEN

    SET ruolo_out = 3;

END IF;

END
```

OPERAZIONI DI INSERT:

Inserisci atleta

Questa procedure inserisce un nuovo atleta nella tabella “atleta”. Viene utilizzato il livello di isolamento READ COMMITTED. Questa procedure fa scattare il trigger “check_lunghezza_cf_atleta”.

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `inserisci_atleta`(in var_cf_atleta
VARCHAR(16), in var_nome VARCHAR(20), in var_cognome VARCHAR(20), in
var_data_nascita DATE)
```

```
BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

    ROLLBACK;

    RESIGNAL;

END;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION

INSERT INTO `personal_trainer_digitale`.`atleta` (
    `cf_atleta`, `nome`, `cognome`, `data_nascita`, `cf_personal`
)
VALUES (
```

```
    var_cf_atleta, var_nome, var_cognome, var_data_nascita, NULL  
);
```

```
COMMIT;
```

```
END
```

Inserisci personal trainer

Questa procedure inserisce un nuovo personal trainer nella tabella “personal_trainer”. Viene utilizzato il livello di isolamento READ COMMITTED. Questa procedure fa scattare il trigger “check_lunghezza_cf_personal”.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `inserisci_personal_trainer`(in  
var_cf_personal VARCHAR(16), in var_nome VARCHAR(20), in var_cognome VARCHAR(20))
```

```
BEGIN
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    RESIGNAL;
```

```
END;
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION;
```

```
INSERT INTO `personal_trainer_digitale`.`personal_trainer` (  
    `cf_personal`, `nome`, `cognome`  
)
```

```
VALUES (  
    var_cf_personal, var_nome, var_cognome  
);
```

```
COMMIT;
```

```
END
```

OPERAZIONI DI SELECT:

Stampa esercizio

La procedura stampa_esercizio consente di visualizzare le informazioni dettagliate relative a un esercizio specifico, identificato dal suo codice. Restituisce dati come nome, descrizione, numero di serie, ripetizioni e, se presenti, le informazioni sul macchinario associato. Utilizza il livello di isolamento READ COMMITTED e garantisce l'integrità attraverso una transazione con gestione degli errori.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `stampa_esercizio`(IN var_codice_es INT)

BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

    ROLLBACK;

    RESIGNAL;

END;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SET TRANSACTION READ ONLY;

START TRANSACTION;

IF NOT EXISTS (
    SELECT 1 FROM dettagli_esercizi WHERE codice_es = var_codice_es
) THEN

    ROLLBACK;

    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Esercizio non trovato!';

ELSE

    SELECT
        nome_esercizio,
        descrizione_esercizio,
        num_serie,
        ripetizioni,
        COALESCE(nome_macchinario, 'Nessun macchinario associato') AS nome_macchinario,
        COALESCE(descrizione_macchinario, 'N/A') AS descrizione_macchinario

    FROM
        dettagli_esercizi

    WHERE
        codice_es = var_codice_es;

    COMMIT;

END IF;

END
```

OPERAZIONI DI DELETE:**Elimina esercizio**

Questa procedura serve ad eliminare un esercizio esistente dalla tabella “esercizi”. Viene utilizzato il livello di isolamento READ COMMITTED.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `elimina_esercizio`(in var_codice_es INT)

BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

    ROLLBACK;

    RESIGNAL;

END;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;

DELETE FROM `personal_trainer_digitale`.`contenuto`

WHERE `codice_es` = var_codice_es;

DELETE FROM `personal_trainer_digitale`.`interagisce`

WHERE `codice_es` = var_codice_es;

DELETE FROM `personal_trainer_digitale`.`macchinario`

WHERE `codice_es` = var_codice_es;

DELETE FROM `personal_trainer_digitale`.`esercizi`

WHERE `codice_es` = var_codice_es;

COMMIT;

END
```

Elimina macchinario:

Questa procedura serve ad eliminare un macchinario esistente dalla tabella “macchinario”. Viene utilizzato il livello di isolamento READ COMMITTED.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `elimina_macchinario`(in var_nome
VARCHAR(20))
```

```
BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

    ROLLBACK;

    RESIGNAL;

END;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

DELETE FROM `personal_trainer_digitale`.`macchinario`

WHERE `nome` = var_nome;

COMMIT;

END
```